ChaCha20-Poly1305 Crypto Core Compatible with Transport Layer Security 1.3

Ronaldo Serrano*, Ckristian Duran*, Trong-Thuc Hoang*†, Marco Sarmiento*,
Akira Tsukamoto†, Kuniyasu Suzaki†‡ and Cong-Kha Pham*
*University of Electro-Communications (UEC), Tokyo, Japan
†National Institute of Advanced Industrial Science and Technology (AIST), Tokyo, Japan
‡Technology Research Association of Secure IoT Edge application based on RISC-V Open architecture (TRASIO)
Email:{ronaldo,duran,thuc,marco}@vlsilab.ee.uec.ac.jp, {akira.tsukamoto,k.suzaki}@aist.go.jp, phamck@uec.ac.jp

2021 18th International SoC Design Conference (ISOCC) | 978-1-6654-0174-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/ISOCC53507.2021.9614016

***Abstract*—The security of the information represents a vital part of all communications protocols. In computer networks, Transport Layer Security (TLS) represents the majority of the use of secure channels for end-to-end communications. However, the efforts are directed only to optimize the software implementations. This paper shows an Authenticated Encryption with Associated Data (AEAD) hardware implementation of ChaCha20-Poly1305 compatible with TLS 1.3. Compared to a software implementation in a RISC-V environment, the performance increase by 7×. The AEAD implementation reaches a speed of 21.5-cycle/byte. The design is implemented in Xilinx Virtex-7 XC7VX485T Field-Programmable Gate Array (FPGA), using 7897 Look-Up Tables (LUT) and 4840 Flip-Flops (FF), represented in 26% of ChaCha20 and 54% of Poly1305.**

***Index Terms*—ChaCha20, Poly1305, TLS, RISC-V**

I. Introduction

Transport Layer Security (TLS) protocol is widespread in computer networks, providing secure channels for end-to-end communications [1]. The Internet Engineering Task Force (IETF) released the standard of TLS 1.3 in 2018 [2]. The standard introduces two security improvements. First, remove the static Rivest Shamir Adleman (RSA) and Diffie-Hellman (DH) cipher suits, encrypting handshake messages. Second, add a ChaCha20 stream cipher with Poly1305 message authentication code in the record layer of the protocol. In this work, we present an architecture of ChaCha20, Poly1305, and the construction of ChaCha20-Poly1305 AEAD compatible with TLS 1.3. In addition, the performance of the AEAD construction is compared with the software implementation using a Rocket core RISC-V processor [3]. The AEAD construction occupies a 7897-LUT and 4840-FF, representing a 30.22% of the resources compared to the processor. The performance increases 7× in comparison to software implementation.

II. ChaCha20 and Poly1305 implementation

Fig. 1.a) shows the implementation of the Chacha20 core. The ChaCha20 implementation is divided into two parts. First, a Block function highlighted in blue takes a 256-bit key, 96-bit nonce, and 32-bit of counter to generate an Initial Matriz (I_M) in the $Initial\ regs$. The states of the I_M are organized in little-endian form. After, the Operation Matriz (O_M) in $Operation\ regs$ obtain the initial states of the $Initial\ regs$. All matrices have 16 states, and each state is a 32-bit register. A Finite State Machine (FSM) highlighted in red runs the 20 rounds in the Quarter Round (QR) operation. The FSM controls the four QR operators' input in each column and diagonal round, respectively. Each round depends only on the result of the previous rounds in O_M. In addition, the $Init$ and $Next$ signals indicate the start of the process and the next 512-bit to processing, respectively. After of 20 rounds the $Matrix\ Adder$ obtain the Final Matrix (F_M), adding the I_M and O_M. Besides, the output of the block function is serialized. Second, a Crypto Function highlighted in green takes the F_M serialized. The 512-bit of $Cipher\ Text$ is obtained by the XOR operation between F_M serialized and the $Plain\ text$. When the Crypto function is finished, a $Valid$ signal is triggered. Fig. 1.b) shows the QR operation implementation. The inputs A, B, C, and D correspond to one stage in O_M, respectively. The QR operations consist of an Add-Rotate-XOR (ARX) cell [4]. The outputs a, b, c, and d return the value in the O_M. The rotate operation highlighted in red is implemented without combinational logic to reduce the delay in the ARX cell, replacing it with a wire permutation.

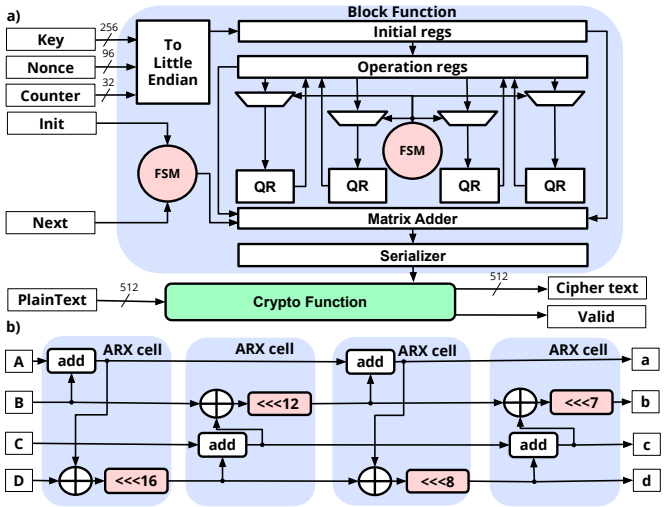

Fig. 1. a) ChaCha20 architecture. b) QR implementation.

Authorized licensed use limited to: UCLA Library. Downloaded on November 17,2022 at 13:33:24 UTC from IEEE Xplore. Restrictions apply.

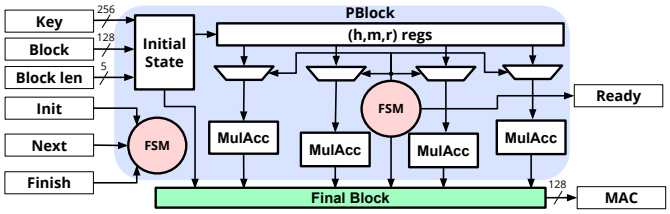

Fig. 2. Poly1305 architecture.

Fig. 2 shows the implementation of the Poly1305 core divided into two parts. First, a PBlock highlighted in blue generates an $Initial\ State$ of r and s, using the 256-bit of the key. After, the 128-bit of $Block$ is operate using the four Multiplier and Accumulator (MulAcc), reproducing the polynomial described in [5]. The length of the message is arbitrary. In this way, the core process 128-bit in each step. The signal $Block\ len$ indicates the numbers of bytes in each $Block$. Besides, the FSM highlighted in red control the input of all $Block$ of the message to authenticate. The signal $Init$ and $Next$ indicates the start of the new message process and a further $Block$ of the message, respectively. In addition, when the last part of the message is introduced, the $Finish$ signal is triggered to run the end MulAcc operation and push the data of the $Final\ Block$. Second, the $Final\ Block$ highlighted in green takes the data accumulation, and s generates the Message-Authentication Code (MAC). The MAC is obtained by adding the s and the data accumulation generated with each MulAcc. Also, the result of the sum is truncated in 128-bit. Finally, the signal $Ready$ indicates the MAC is generated.

III. AEAD CONSTRUCTION

The standard determines AEAD construction using a 256-bit key, 96-bit nonce, an arbitrary length plaintext, and arbitrary-length Additional Authenticated Data (AAD) [5]. Fig. 3 illustrates the implementation of AEAD peripheral. The key used in Poly1305 in TLS 1.3 is generated by ChaCha20, using the same key and nonce of the keystream. The ChaCha20 and Poly1305 are implemented in the same peripheral to reduce movement time between the respective cores. An FSM highlighted in red control the signal of ChaCha20 and Poly1305 cores in the peripheral. The ChaCha20 core generates the different parts of the ciphertext, while the Poly1305 processes the different blocks simultaneously. The different parts of the block construction only processing 16-byte with the Poly1305 core simultaneously. A 2-bit $Control$ signal indicates when the next block is updated and the last block of the AAD.

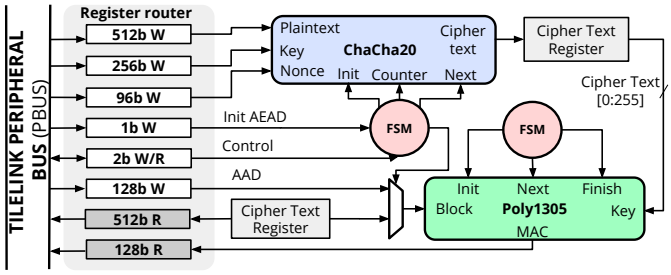

Fig. 3. AEAD peripheral architecture.

TABLE I
PERFORMANCE SUMMARY AND COMPARISON IN FPGA.

	This work				[4]
Module	**ChaCha20**	**Poly1305**	**AEAD**	**Software Rocket**	**ChaCha20**
LUT	2288	5044	7897	29477	2369
FF	1050	1856	4840	12659	2152
Total	3338	6900	12737	42136	4521
Cycles/Byte	0.34	1.22	21.5*	156	0.53

*Process and movement data time.

IV. RESULTS

The performance of the AEAD implemented is compared with software, using a Rocket core with 16-KB of instruction-cache and data cache. Table I shows the results of the ChaCha20-Poly1305 implementation in FPGA. The ChaCha20 core occupies a 2288-LUT and 1050-FF with 0.34-cycle/byte. The related work uses to comparison proposes other architecture of ChaCha20 with more performance. However, the resources represent three times more of all AEAD construction. Poly1305 core occupies a 5044-LUT and 1856-FF with 1.22-cycle/byte. Finally, the AEAD construction occupies a 7897-LUT and 4840-FF with 21.5-cycle/byte. The performance of the AEAD construction presented taking into account the cycles of movement of the data, using a TileLink bus [6]. The cycles of movement of the key, nonce, AAD, the plaintext, and the control signal used in the AEAD construction represent a 78.26% of all cycles.

V. CONCLUSION

A ChaCha20, Poly1305, and ChaCha20-Poly1305 AEAD construction compatible with TLS 1.3 are presented in this paper. The implementation was done on a Xilinx Virtex-7 XC7VX485T FPGA. The performance of the architecture implemented is 21.5-cycle/byte with the time of the data movement. The implementation occupies a 7897-LUT and 4840-FF, representing a 30.22% of a Rocket core. The performance increases a 725.68% in comparison between the software implementation.

ACKNOWLEDGMENT

This work is based on results obtained from project JPNP16007, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

[1] J. Li *et al.*, "ME-TLS: Middlebox-Enhanced TLS for Internet-of-Things Devices," *IEEE Internet of Things Journal*, vol. 7, no. 2, pp. 1216–1229, 2020.

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: https://rfc-editor.org/rfc/rfc8446.txt

[3] RISC-V Foundation, "Rocket Chip Generator," 2019. [Online]. Available: https://github.com/chipsalliance/rocket-chip

[4] J. Pfau *et al.*, "A Hardware Perspective on the ChaCha Ciphers: Scalable Chacha8/12/20 Implementations Ranging from 476 Slices to Bitrates of 175 Gbit/s," in *2019 32nd IEEE International System-on-Chip Conference (SOCC)*, 2019, pp. 294–299.

[5] Y. Nir *et al.*, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, Jun. 2018. [Online]. Available: https://rfc-editor.org/rfc/rfc8439.txt

[6] SiFive, Inc., "SiFive TileLink Specication," Aug. 2019. [Online]. Available: https://www.sifive.com/documentation/tilelink/tilelink-spec/

Authorized licensed use limited to: UCLA Library. Downloaded on November 17,2022 at 13:33:24 UTC from IEEE Xplore. Restrictions apply.